

# A Serverless Fog Computing Architecture for Smart Waste Management: Design, Deployment, and Evaluation on AWS

Gundeti Sachin Reddy  
*Master of Science in Cloud Computing*  
*National College of Ireland*  
Dublin, Ireland  
x23432721@student.ncirl.ie

**Abstract**—This paper presents the design, implementation, and real-world cloud deployment of a fog and edge computing pipeline for smart municipal waste management. The system simulates five distinct sensor types (bin fill level, internal temperature, gas concentration, bin weight, and lid-open events) across two collection districts, each independently configurable for sampling and dispatch frequency. A Node.js fog node performs windowed aggregation and threshold-based alerting before dispatching processed batches to a scalable cloud backend built from Amazon SQS, AWS Lambda, and Amazon DynamoDB. The dashboard, originally hosted on a persistent EC2 instance, was subsequently migrated to a fully serverless architecture (Amazon S3, AWS Lambda, and Amazon API Gateway) to align the entire backend with the module's function-as-a-service scalability criterion and to resolve a genuine campus-network access restriction. The complete system was deployed and tested on a real AWS account rather than a local emulator alone, which surfaced defects invisible to local testing, including credential-handling bugs that only manifest under real temporary AWS credentials, and undocumented platform restrictions specific to the AWS Academy Learner Lab environment. This paper critically analyses the architectural decisions taken, the defects discovered during real deployment, and the lessons learned for building genuinely cloud-native fog computing systems.

**Keywords**—fog computing, edge computing, Internet of Things, serverless architecture, AWS Lambda, cloud deployment, smart waste management

## I. INTRODUCTION

Municipal waste collection is traditionally scheduled on a fixed calendar rather than on actual bin conditions, which routinely results in either premature collection runs to bins that are not yet full or, conversely, overflowing bins and associated fire and odor hazards that are only discovered after the fact. Internet of Things sensing, combined with fog and edge processing close to the data source, allows a waste authority to move from a fixed schedule to a condition-driven one, collecting the busiest bins first while still processing data efficiently rather than streaming every raw reading to the cloud.

This paper documents the design and construction of a fog and edge computing solution for this domain, built as an individual assignment for the Fog and Edge Computing (H9FECC) module. The brief for this coursework required, at minimum: (1) a sensor and fog layer generating data from three to five distinct sensor types with independently configurable sampling and dispatch rates, where the fog node genuinely receives and processes sensor data before

dispatching it onward; (2) a scalable backend layer, designed around mechanisms such as queues, function-as-a-service (FaaS), or autoscaling, that processes the fog node's output and exposes a responsive dashboard for the sensor types; and (3) deployment and testing of that backend on a public cloud platform, not merely a local simulation.

The remainder of this paper is organised as follows. Section II describes the system's architecture and the design decisions taken at each layer, including a mid-project migration of the dashboard to a fully serverless model. Section III describes the concrete implementation: the software components and libraries used, the automated test suite, the continuous integration and deployment pipeline, and the real AWS deployment together with the genuine defects it surfaced. Section IV concludes with a critical reflection on the project and directions for future work.

## II. SYSTEM ARCHITECTURE AND DESIGN

### A. Sensor and Fog Layer

The simulated deployment models two waste-collection districts, district-a and district-b, each instrumented with five sensor types: fill\_level\_pct, internal\_temp\_c, gas\_level\_ppm, bin\_weight\_kg, and lid\_open\_count. Every one of the ten resulting sensor processes (five types across two districts) exposes two independent, separately configurable environment variables, SAMPLE\_INTERVAL and DISPATCH\_INTERVAL, satisfying the brief's requirement that generation frequency and dispatch rate be genuinely decoupled knobs rather than a single aliased value. Each sensor process was verified to hold a pairwise-distinct combination of these two values, since a duplicated pair would otherwise silently fail to demonstrate that the two knobs are truly independent.

The fog node is a Node.js HTTP service that ingests raw readings over its own REST endpoint, buffers them per sensor type and site, and performs genuine windowed aggregation, not a pass-through relay, computing a minimum, maximum, average, latest value, and reading count over a fixed time window. Each aggregated window is evaluated against a small set of domain-specific threshold rules (for example, a fill level exceeding a configured percentage raises a collection\_needed alert, and an internal temperature exceeding a configured threshold raises a fire\_risk\_warning), and the resulting aggregate-plus-alerts payload is dispatched as a single message to an Amazon SQS queue. Performing aggregation and alerting in the fog layer, rather than forwarding every raw reading to the cloud, is the central architectural argument for

calling this layer “fog” rather than a simple network relay: it reduces the volume and frequency of cloud-bound traffic while still preserving the information a backend consumer needs.

### *B. Scalable Backend Layer*

The backend layer is built entirely from managed, scalable AWS primitives rather than a hand-rolled server: an Amazon SQS standard queue decouples the fog node's dispatch rate from the backend's processing rate, and an AWS Lambda function, subscribed to that queue via an event-source mapping, is invoked automatically as messages arrive, writing each aggregated window into an Amazon DynamoDB table. This queue-plus-FaaS pairing is one of the scalability mechanisms the brief explicitly names as an example, and it was chosen specifically because it requires no server capacity planning: the queue absorbs bursts, and Lambda's own concurrency model scales the number of concurrent invocations to the backlog without any manual provisioning.

The DynamoDB table uses `sensor_type` as its partition key and a composite sort key of the form `window_end#site_id` as its range key. This design choice is deliberate: if the sort key were `window_end` alone, two districts producing an aggregate for the same sensor type in the same flush cycle would collide on an identical primary key, and whichever write landed second would silently overwrite the first, permanently losing one district's reading for that window. Appending the site identifier to the sort key disambiguates the two districts' concurrent writes without requiring any additional index, and was verified directly by confirming that both districts' aggregates for a shared window are retained independently rather than one clobbering the other.

### *C. Dashboard Layer and Its Migration to a Fully Serverless Architecture*

The dashboard was originally implemented as a fourth long-running container alongside the fog node and the ten sensor processes, exposing a small REST API and a static single-page frontend from one Node.js HTTP server on a dedicated port of the same EC2 instance used for the fog node. This was a pragmatic first deployment, but on critical review it had two weaknesses. First, it was the one piece of the “scalable backend” requirement still resting on a persistent server rather than on a queue or FaaS mechanism, unlike every other backend component. Second, and more concretely, a raw EC2 public IP address on a non-standard port served over plain HTTP is exactly the traffic shape that campus and corporate WiFi networks commonly block by policy; this was confirmed as a genuine, reported access failure rather than a hypothetical concern.

The dashboard was therefore migrated to a fully serverless model: its static assets (HTML, CSS, and JavaScript) are now served directly from an Amazon S3 bucket over the bucket's own HTTPS endpoint, and its small REST API was extracted into its own AWS Lambda function, reusing the existing route-handling logic unchanged through a thin adapter that translates between the API gateway's event format and the original Node.js request/response shapes. Two platform-specific constraints of the AWS Academy Learner Lab account used for this deployment were discovered empirically during this migration and are worth recording as design evidence: Amazon CloudFront is unavailable in this account, including read-only operations, so the S3 bucket is served over its own virtual-hosted-style HTTPS endpoint rather than through a CloudFront distribution; and public, unauthenticated AWS Lambda Function URLs are also

blocked, confirmed by a successful authenticated invocation of the same function returning a Forbidden response when invoked through its Function URL. Amazon API Gateway was not subject to either restriction and was used instead as the public entry point in front of the dashboard's Lambda function.

The fog node and the ten sensor processes remain on the EC2 instance. This was a deliberate, and not merely residual, architectural decision: both are continuous, long-running processes rather than naturally invoke-on-demand units of work, which makes them a poor fit for Lambda's request-driven execution model, and neither is ever reached directly by a browser, so the campus-network restriction that motivated moving the dashboard does not apply to them.

### *D. Deployment Topology*

Fig. 1 summarises the resulting topology. Sensor processes and the fog node run in Docker containers on a single EC2 instance, managed entirely through AWS Systems Manager Session Manager rather than SSH, so no key pair is provisioned and no inbound SSH port is opened on the instance's security group. The fog node dispatches aggregated windows to an SQS queue, which triggers a Lambda function that writes to a DynamoDB table. A second Lambda function, sitting behind an API Gateway HTTP API, reads from that same table (and from the queue and the first Lambda function's own metadata) to answer the dashboard's REST API calls, while the dashboard's static frontend is served independently from Amazon S3. This topology places every backend component except the sensor and fog tier behind a managed, serverless service boundary.

## III. IMPLEMENTATION

### *A. Software Components and Libraries*

The sensor, fog, processor, and dashboard components are all implemented in Node.js 20, using the official AWS SDK for JavaScript v3 (`@aws-sdk/client-sqs`, `@aws-sdk/client-dynamodb`, `@aws-sdk/lib-dynamodb`, and `@aws-sdk/client-lambda`) for every AWS interaction, and Node's own built-in `http` module rather than a web framework such as Express, keeping the dependency surface deliberately small. The dashboard's trend visualisation is rendered client-side with `Chart.js`, vendored locally rather than loaded from a content-delivery network so the page has no third-party runtime dependency. Local development and continuous integration both use Docker Compose together with LocalStack, an open-source AWS cloud emulator, so the same SQS, DynamoDB, and Lambda interactions used in production can be exercised without touching a real AWS account during day-to-day development.

### *B. Testing Strategy*

Every module (sensors, fog, backend processor, and backend dashboard) has its own automated test suite written with Node's built-in test runner (`node:test`), exercised through hand-written fake AWS client objects rather than mocking libraries, so that each unit test asserts on the exact request shape sent to a given AWS SDK call. Beyond unit tests, a load-testing script bursts a configurable number of synthetic messages directly onto the real queue and asserts, first, that the queue depth is non-zero immediately after the burst (proving the messages genuinely reached SQS rather than being silently dropped), and second, that the queue depth has either fully drained or measurably decreased within a timeout window, allowing for the underlying Lambda concurrency ceiling without treating a slower-than-expected drain as a hard failure.

This two-tier assertion pattern was deliberately chosen so that the load test remains a meaningful elasticity check without becoming flaky under normal, expected throughput variation.

### C. Continuous Integration and Deployment

A GitHub Actions workflow runs the full automated test suite for all four modules, then brings the entire stack up with Docker Compose against LocalStack and exercises the live pipeline end-to-end, on every push to the repository. A second, separately triggered GitHub Actions workflow exists specifically to restart the EC2-hosted fog and sensor tier after an AWS Academy Learner Lab session ends and a fresh one begins, since the Learner Lab platform stops the EC2 instance between sessions; this workflow reads its AWS credentials from encrypted GitHub repository secrets rather than accepting them as plain workflow inputs, verifies that the credentials resolve to the expected AWS account before doing anything else, and only then restarts the instance and re-verifies the pipeline.

### D. Real AWS Deployment and Defects Discovered

The complete system, including the sensor and fog tier, the SQS queue, both Lambda functions, the DynamoDB table, the API Gateway endpoint, and the S3-hosted frontend, was deployed to a real AWS account (an AWS Academy Learner Lab account) in region us-east-1, satisfying the brief's requirement that the backend be deployed and tested on an actual public cloud platform rather than only a local emulator. This real deployment was directly responsible for surfacing three classes of defect that the local, LocalStack-backed test suite had not caught.

First, three separate files across the processor and dashboard components constructed their AWS SDK client credentials using a check that was true in both the local and the real-AWS case: whenever an AWS access key environment variable was present, the code applied a hardcoded, LocalStack-only static credential pair. Because real AWS Lambda and EC2 execution environments always populate that same environment variable automatically, with genuinely temporary, session-token-bearing credentials, this logic silently discarded the real temporary credentials in favour of the hardcoded pair on every real deployment, causing every DynamoDB and SQS call to fail authentication in production despite passing every LocalStack-backed test. The fix was to gate the hardcoded override on the presence of an explicit LocalStack endpoint override instead, which is the only condition that is actually unique to the local-emulator case.

Second, the dashboard's Lambda adapter initially passed through a numeric Content-Length header value copied from the original Node.js response object; AWS Lambda's HTTP response format requires every header value to be a string, and a numeric value was silently rejected, producing a generic "Internal Server Error" with no corresponding error recorded in the function's own execution logs, since the function itself completed successfully and only its response shape was rejected downstream. Third, API Gateway's own "quick create" tooling, when pointed at an existing Lambda function, did not actually attach the resource-based invoke permission it documents as automatic, which was only discovered by comparing a successful direct, authenticated Lambda invocation against a failing request through the newly created API Gateway endpoint.

All three defects were fixed and re-verified against the live deployment, including a live fault-injection check in which the underlying dependency was deliberately made unavailable to

confirm the fix degrades gracefully rather than merely passing under normal conditions. The source code, including the fixes described above, is available at [https://github.com/valletivarish/FEC\\_version\\_2](https://github.com/valletivarish/FEC_version_2), under projects/22-smart-waste-management, together with a readme.txt documenting installation, configuration, and the deployment history described in this section.

## IV. CONCLUSION

This project set out to build a complete fog and edge computing pipeline for smart waste management, satisfying the module brief's requirements for a configurable sensor and fog layer, a scalable cloud backend, and genuine deployment and testing on a public cloud platform, and to critically analyse the architectural decisions taken at each layer. All three layers were implemented, tested, and deployed to a real AWS account rather than a local simulation alone.

The most significant finding of this project is not architectural but methodological: local testing against a cloud emulator, however thorough, is not a substitute for testing against the real target platform. Every defect discovered during the real AWS deployment, the credential-handling bug, the Lambda response-format bug, and the API Gateway permission gap, passed the full local test suite and a full LocalStack-backed integration test cleanly, and would have shipped undetected had the project stopped at the local-simulation stage. This directly reinforces the module's own assessment criterion that each layer be "developed, tested, and deployed" as three distinct activities, not a single combined one.

The most difficult part of this project was not writing application code but discovering the AWS Academy Learner Lab account's own platform restrictions, several of which (CloudFront being entirely blocked, and public Lambda Function URLs being blocked while API Gateway is not) are not documented anywhere and were only discoverable by attempting each service directly and reading the resulting access-denied errors. Future work would include converting the fog and sensor tier itself to a fully event-driven, scheduled-Lambda model rather than a continuously running EC2 instance, and, if the account restrictions were lifted, reintroducing a CloudFront distribution in front of the S3-hosted frontend for a custom domain and edge caching.

## REFERENCES

- [1] Amazon Web Services, "AWS Lambda Developer Guide," Amazon Web Services, Inc. [Online]. Available: <https://docs.aws.amazon.com/lambda/>
- [2] Amazon Web Services, "Amazon DynamoDB Developer Guide," Amazon Web Services, Inc. [Online]. Available: <https://docs.aws.amazon.com/dynamodb/>
- [3] Amazon Web Services, "Amazon Simple Queue Service Developer Guide," Amazon Web Services, Inc. [Online]. Available: <https://docs.aws.amazon.com/sqs/>
- [4] Amazon Web Services, "Amazon API Gateway Developer Guide," Amazon Web Services, Inc. [Online]. Available: <https://docs.aws.amazon.com/apigateway/>
- [5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in Proc. 1st ACM Mobile Cloud Computing Workshop (MCC '12), Helsinki, Finland, 2012.
- [6] Docker Inc., "Docker Compose overview," Docker Documentation. [Online]. Available: <https://docs.docker.com/compose/>
- [7] OpenJS Foundation, "Node.js Documentation." [Online]. Available: <https://nodejs.org/docs/>
- [8] LocalStack, "LocalStack Documentation." [Online]. Available: <https://docs.localstack.cloud/>